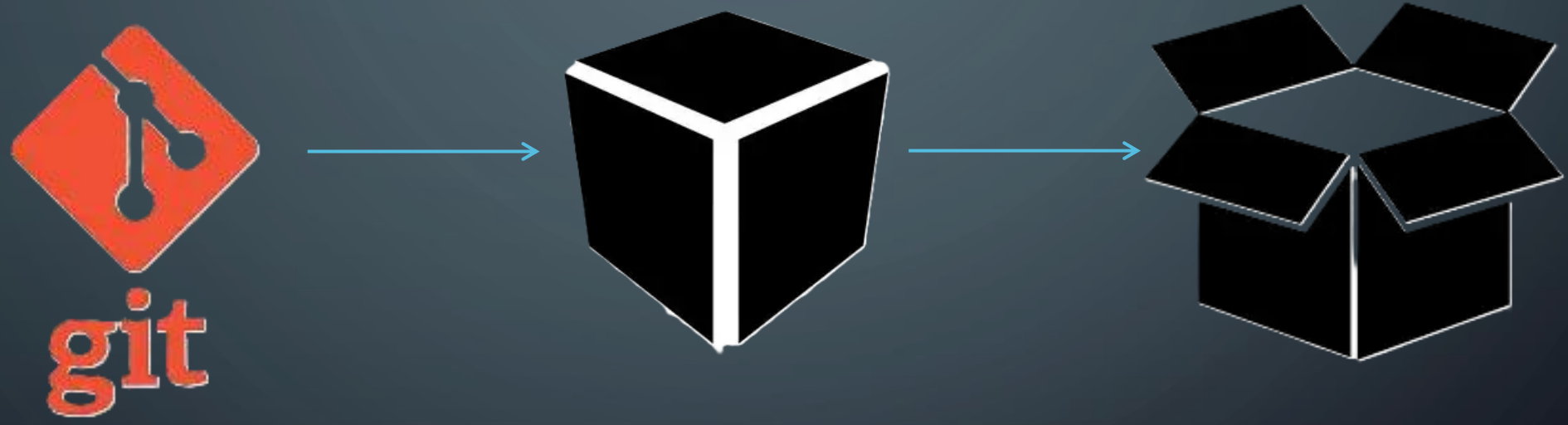
# B.I.G

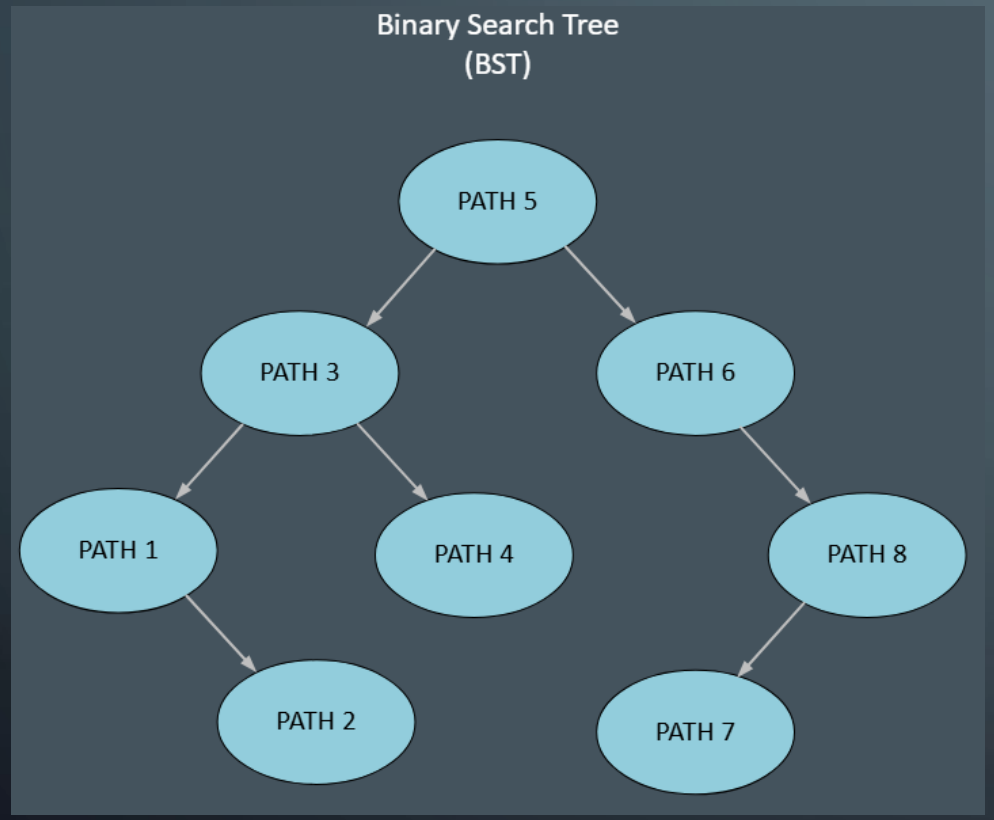
A BST-Indexed Graph Model for Version Control

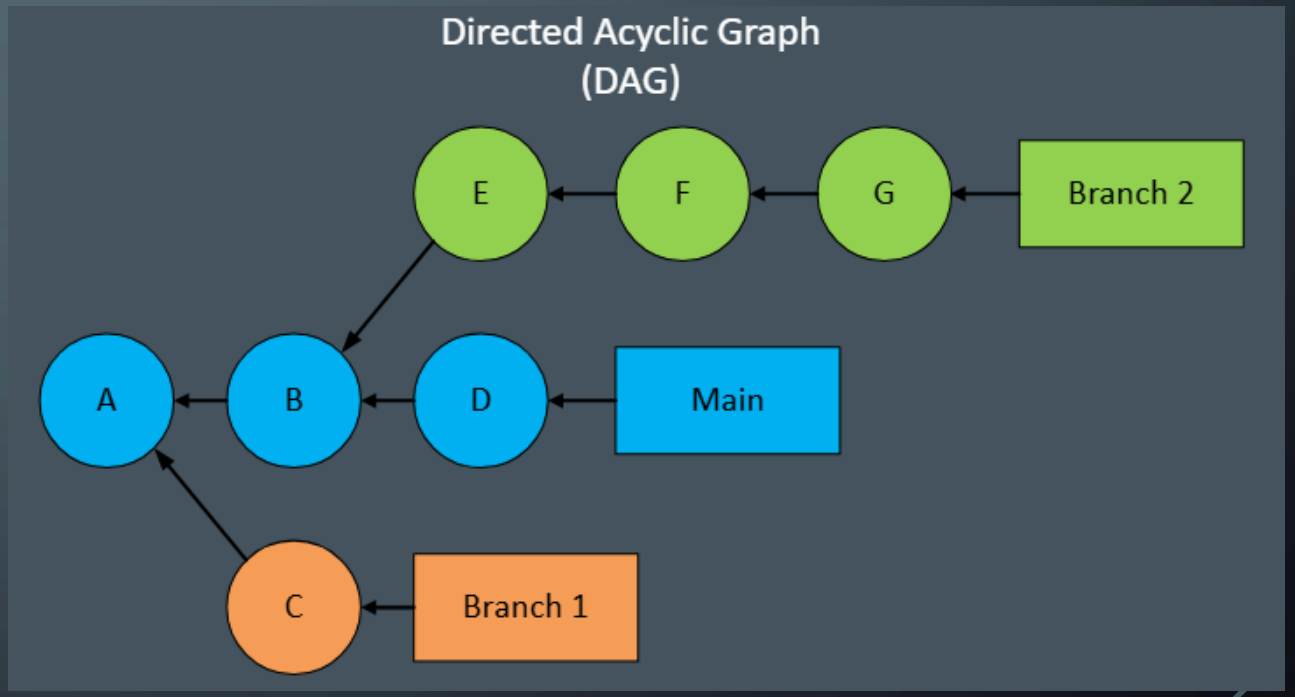
Final project in Data Structures course

# Motivation

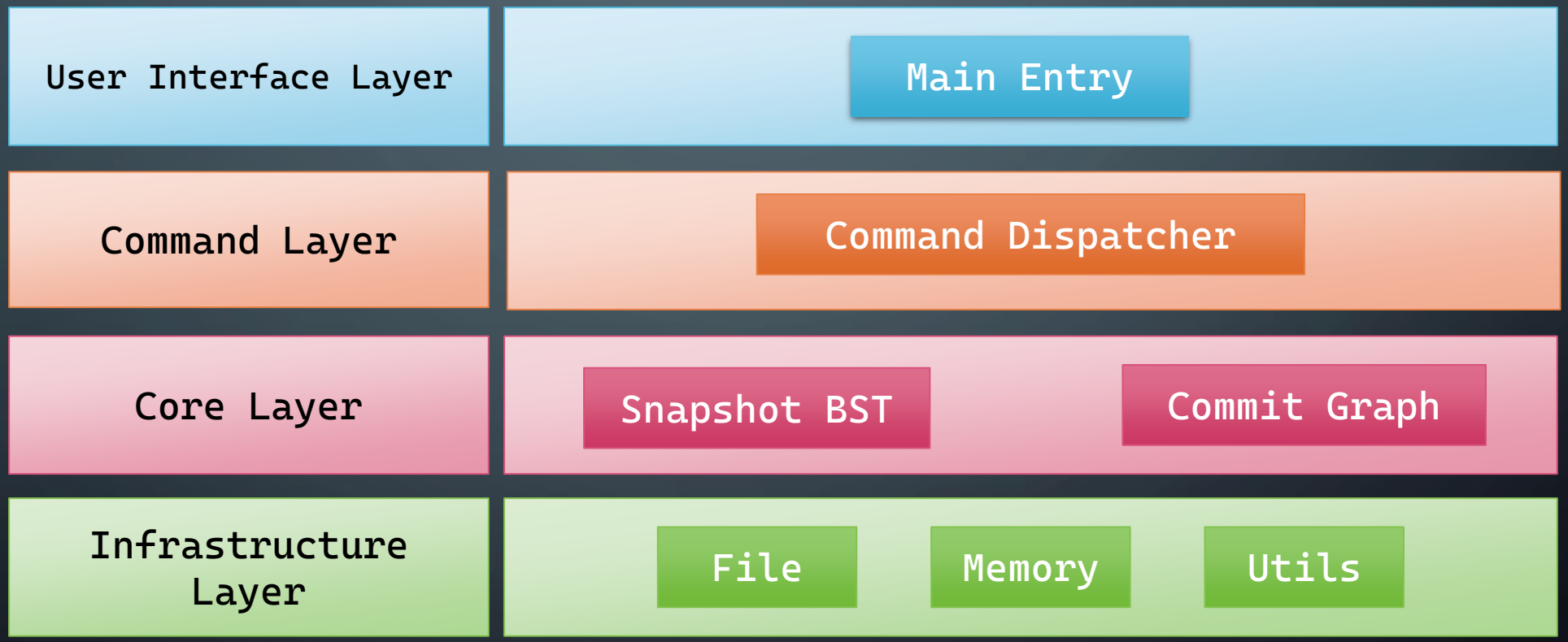


# Methodology



# Methodology

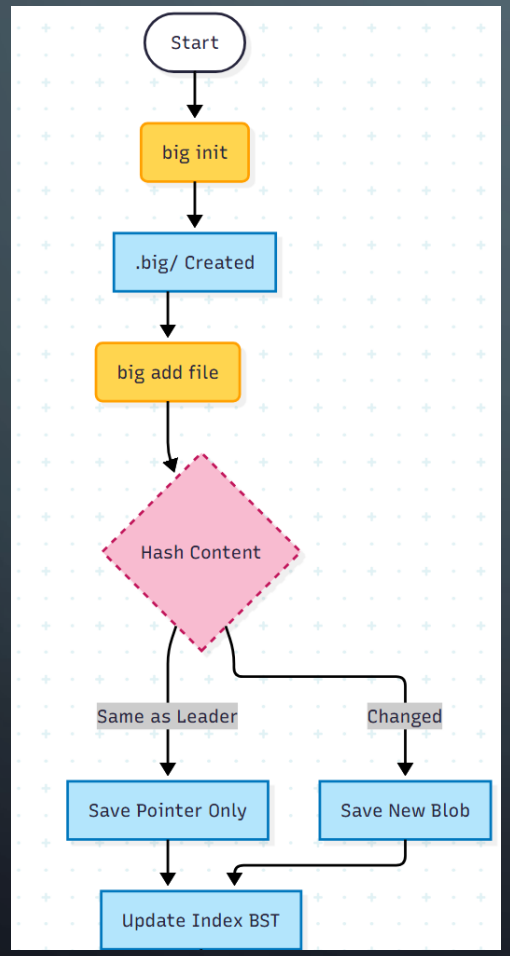


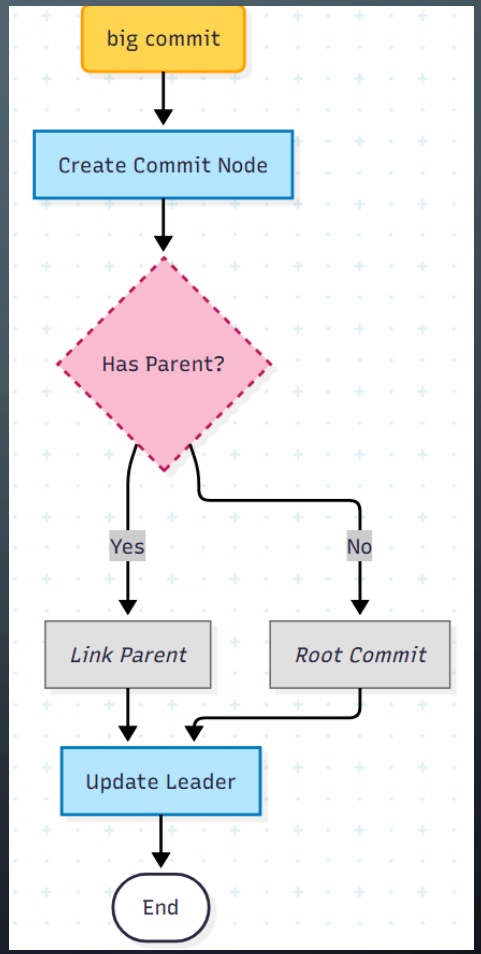
# Development Environment

- **OS/Kernel**: Windows 11 with **WSL2 (Ubuntu)**
- **Language**: C (C17 Standard)
- **Compiler/Build**: GCC, Makefile
- **Tools**: VSCode, Valgrind, Git

# Proposed Features – Phase 1 (The Core)

## init -> add -> commit (The Core Workflow)



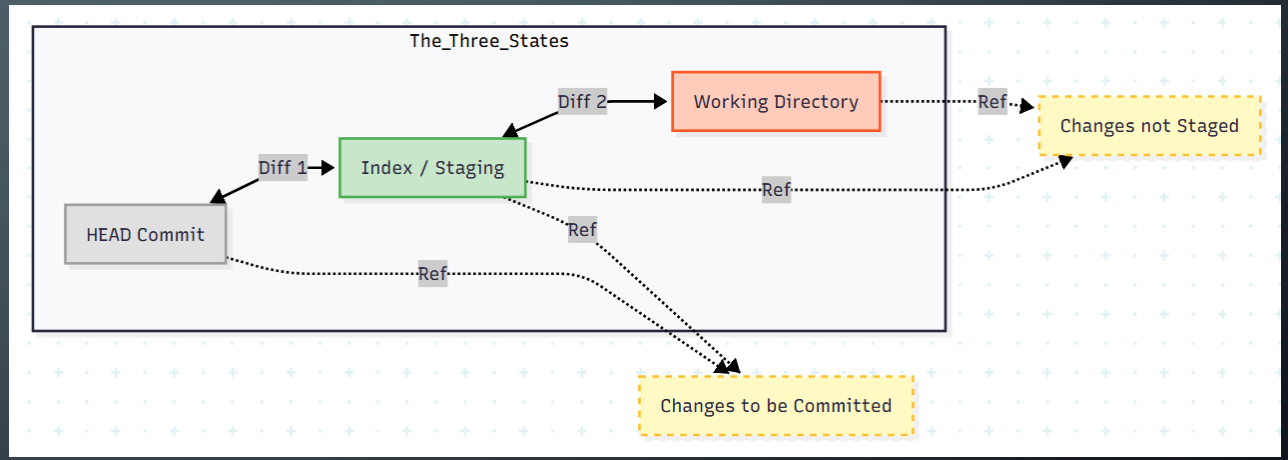



# Phase 2 (Visibility)

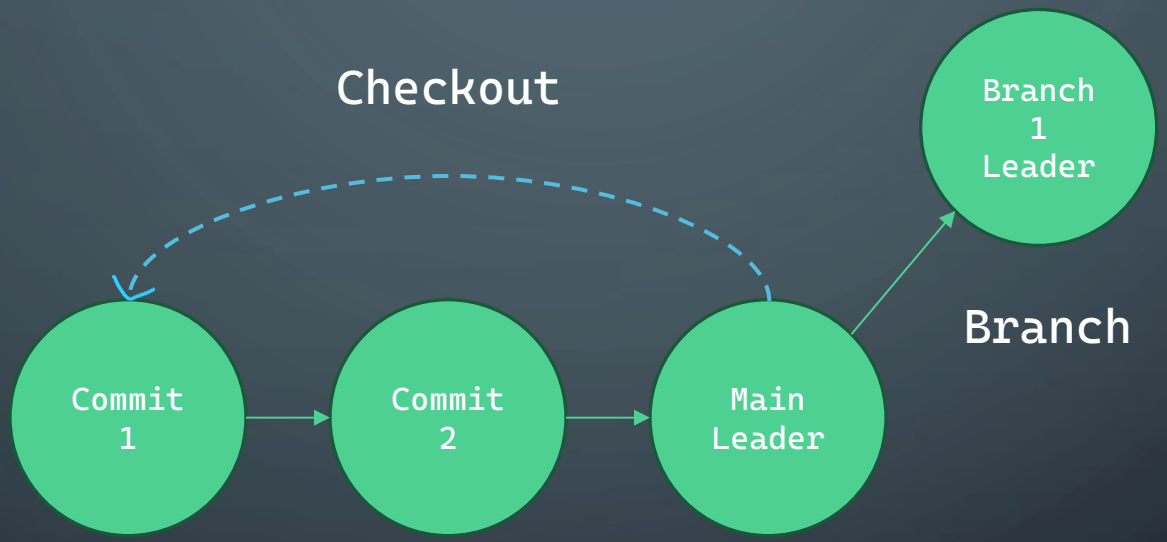
## Log

```
306a8d5 feat: Implemnt 'big add' tracing commit path is content is same
048a3cb feat: 'big add' now can trace Leader and compare content
803db40 feat: Implement new 'big commit' with new 'big add' feature
2f10f74 docs: Add comments in main.c and utils/ code
3452e72 feat: Rewrite 'big add' feature creating a index directory include all files
0c90c65 feat: Implement 'big status' feature of tracing files are in index or not
189081a refactor: Replace malloc, free and fopen with xmalloc, xfree and xfopen
44d1f25 feat: Implement fopen wrapper
cf5ed60 feat: Implement malloc and free wrapper
986f2da build: More strict compiler check
```

## Status



# Phase 3 (Navigation)



# Conclusion

Feature list:

- init
- add
- commit
- log
- checkout
- branch

What I will learn:

- System Programming (File I/O, Process Control).
- Memory Management (Manual Malloc/Free).
- Data Structures in Practice (BST + DAG).